

Laboratorio: recursión y backtracking

MOMENTO 1 — Enunciado del Laboratorio 3

El caso: rutas de recolección en un barrio. Tienen un mapa como matriz, donde cero es calle transitable, uno es calle bloqueada, y hay puntos de acopio marcados con dos. El camión entra por la esquina superior izquierda.

- **Parte A — El camino básico (35%)**

Encuentren un camino desde la entrada hasta la esquina inferior derecha, esquivando los bloqueos, e impriman el mapa con el camino marcado. Esto es el laberinto del lunes, adaptado.

- **Parte B — Recolección obligatoria (40%)**

Ahora el que hace pensar. El camión no puede simplemente llegar a la salida: tiene que pasar por todos los puntos de acopio antes de salir. Encuentren un camino que los recoja todos.

Piénsenlo antes de programar. El estado ya no es solo dónde estoy: es dónde estoy y cuántos puntos llevo recogidos. El caso base cambia: llegar a la salida ya no basta, hay que llegar con todos recogidos.

- **Parte C — Análisis (25%)**

En el archivo de respuestas: cuántos caminos distintos encontró su algoritmo en la Parte A; qué pasa con el tiempo de ejecución cuando el mapa crece de cinco por cinco a diez por diez, medido de verdad, no estimado; y una explicación en cuatro líneas de por qué este problema no se puede resolver bien con ciclos anidados.

Como siempre, los dos lenguajes. Y en C++ compilen con el sanitizer, aunque hoy no haya memoria dinámica: una recursión desbordada también la detecta.

MOMENTO 2 — Desarrollo

- **PARTE A** Si un equipo dice «se cuelga», casi siempre es que no está marcando las casillas visitadas, y da vueltas en círculo. Si dice «no encuentra salida cuando sí hay», es que no está desmarcando.
- **PARTE B** El atasco es de diseño: intentan resolverlo con la misma función de la Parte A. La pregunta que desatasca es «¿qué información necesita llevar la función además de la posición?».
- **PARTE C** Recuérdeles medir de verdad, con la librería de tiempo, no estimar. Es la primera vez que miden y les cuesta.

Antes de arrancar, dos consejos. Uno: empiecen imprimiendo el mapa en cada llamada recursiva, aunque ensucie la salida. Ver por dónde va el algoritmo es la forma más rápida de encontrar el error, y después borran los prints.

Y dos: prueben primero con un mapa de tres por tres que hayan resuelto a mano. Si el algoritmo falla ahí, no van a poder depurarlo en uno de diez por diez.

```

MAPA 5x5
Parte A - Cantidad de caminos distintos hacia la salida: 16
Parte B - Cantidad de caminos distintos con recolección completa: 4
Ruta más rápida y eficiente (menor cantidad de pasos):
. . . . 0
. . 1 . .
0 0 1 . .
1 0 0 . 1
0 0 1 . .

Pasos mínimos requeridos: 12
Tiempo real de ejecución: 0.000915 segundos

```

```

MAPA 10x10
Parte A - Cantidad de caminos distintos hacia la salida: 3168
Parte B - Cantidad de caminos distintos con recolección completa: 1440
Ruta más rápida y eficiente (menor cantidad de pasos):
. 0 . . . . . 0 0
. . . 1 0 0 1 . . 0
0 0 1 0 0 1 0 0 . 0
1 0 0 0 1 0 0 1 . 0
0 1 0 1 0 0 1 0 . .
0 0 0 0 1 1 1 0 1 .
0 1 0 1 0 0 1 1 0 .
0 1 0 0 1 0 1 0 1 .
0 0 0 1 0 0 1 0 . .
0 1 0 1 0 1 0 0 . .

Pasos mínimos requeridos: 22
Tiempo real de ejecución: 0.105575 segundos

```

Esta solución funciona tan bien porque analiza absolutamente todas las calles y cruces posibles antes de tomar una decisión. Al recordar en cada paso cuántos puntos de basura lleva recogidos, asegura que el camión no se dirija a la salida antes de tiempo. Además, como va anotando en un mapa las calles por las que ya pasó, evita que el camión se devuelva o dé vueltas en círculo, garantizando así la ruta más corta y eficiente posible.